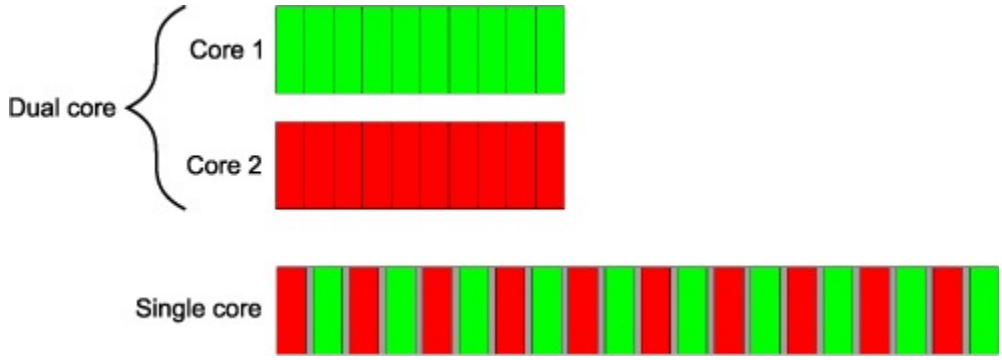
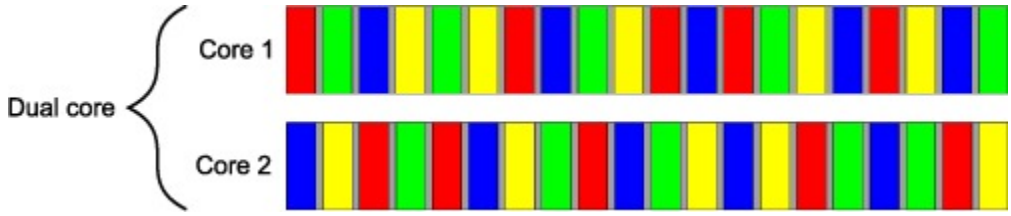


Though the availability of concurrency in the hardware is most obvious with multiprocessor or multicore systems, some processors can execute multiple threads on a single core. The important factor to consider is really the number of *hardware threads*: the measure of how many independent tasks the hardware can genuinely run concurrently. Even with a system that has genuine hardware concurrency, it's easy to have more tasks than the hardware can run in parallel, so task switching is still used in these cases. For example, on a typical desktop computer there may be hundreds of tasks running, performing background operations, even when the computer is nominally idle. It's the task switching that allows these background tasks to run and allows you to run your word processor, compiler, editor, and web browser (or any combination of applications) all at once. Figure 1.2 shows task switching among four tasks on a dual-core machine, again for an idealized scenario with the tasks divided neatly into equalsize chunks. In practice, many issues will make the divisions uneven and the scheduling irregular. Some of these issues are covered in chapter 8 when we look at factors affecting the performance of concurrent code.

Figure 1.2. Task switching of four tasks on two cores



All the techniques, functions, and classes covered in this book can be used whether your application is running on a machine with one single-core processor or on a machine with many multicore processors and are not